

TypeScript Interview Questions / Cau hoi phong van TypeScript

1. What are the basic types in TypeScript? / Cac kieu du lieu co ban trong TypeScript la gi?

EN: TypeScript has primitive types: `string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`. Also `any`, `unknown`, `never`, `void` for special cases.

```
let name: string = "John";
let age: number = 30;
let isActive: boolean = true;
```

VI: TypeScript co cac kieu nguyen thuy: `string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`. Them `any`, `unknown`, `never`, `void` cho cac truong hop dac biet.

2. How to define array and tuple types? / Cach dinh nghia kieu array va tuple?

EN: Arrays use `type[]` or `Array<type>`. Tuples are fixed-length arrays with specific types at each position.

```
let numbers: number[] = [1, 2, 3];
let items: Array<string> = ["a", "b"];
let tuple: [string, number] = ["hello", 42]; // fixed length & types
```

VI: Array dung `type[]` hoac `Array<type>`. Tuple la mang co do dai co dinh voi kieu cu the tai moi vi tri.

3. What is enum and when to use it? / Enum là gì và khi nào nên dùng?

EN: Enum defines a set of named constants. Use for fixed sets of related values like status codes, directions, roles.

```
enum Status { Pending, Active, Completed }
enum Direction { Up = "UP", Down = "DOWN" }

let status: Status = Status.Active; // 1
let dir: Direction = Direction.Up; // "UP"
```

VI: Enum định nghĩa tập hợp các hằng số có tên. Dùng cho các tập giá trị có định liên quan như mã trạng thái, hướng, vai trò.

4. What is the difference between interface and type alias? / Sự khác biệt giữa interface và type alias?

EN: Both define object shapes, but:

- Interface: extendable, can be merged (declaration merging)
- Type: can represent unions, primitives, tuples, mapped types

```
interface User { name: string; }
interface User { age: number; } // merged

type ID = string | number; // union - only type can do this
```

VI: Cả hai đều định nghĩa hình dạng object, nhưng:

- Interface: có thể mở rộng, có thể gộp (declaration merging)
- Type: có thể biểu diễn union, primitive, tuple, mapped types

5. When should you use interface over type alias? / Khi nao nen dung interface thay vi type alias?

EN: Use interface for:

- Object shapes that might be extended
- Public API contracts (libraries)
- Class implementations

Use type for unions, intersections, mapped types, or complex type manipulations.

VI: Dung interface khi:

- Object co the duoc mo rong
- Contract API cong khai (thu vien)
- Class implementations

Dung type cho union, intersection, mapped types, hoac cac thao tac kieu phuc tap.

6. Can interface extend type and vice versa? / Interface co the extend type va nguoc lai khong?

EN: Yes, they can interoperate:

```
type Animal = { name: string };  
interface Dog extends Animal { breed: string; } // interface extends type  
  
interface Person { name: string; }  
type Employee = Person & { company: string }; // type intersects interface
```

VI: Co, chung co the tuong tac voi nhau: interface co the extend type, va type co the intersect interface.

7. What are union types? / Union types là gì?

EN: Union types allow a value to be one of several types. Use `|` operator.

```
type ID = string | number;
function printId(id: ID) {
  if (typeof id === "string") {
    console.log(id.toUpperCase()); // string methods available
  } else {
    console.log(id.toFixed(2)); // number methods available
  }
}
```

VI: Union types cho phép giá trị thuộc một trong nhiều kiểu. Dùng toán tử `|`. Cần kiểm tra kiểu trước khi dùng method cụ thể.

8. What are intersection types? / Intersection types là gì?

EN: Intersection types combine multiple types into one. Use `&` operator. The result has all properties from all types.

```
type Person = { name: string };
type Employee = { employeeId: number };
type Staff = Person & Employee;

const staff: Staff = { name: "John", employeeId: 123 }; // must have both
```

VI: Intersection types kết hợp nhiều kiểu thành một. Dùng toán tử `&`. Kết quả có tất cả properties từ các kiểu.

9. What are literal types? / Literal types là gì?

EN: Literal types are exact values as types. Useful for restricting values to specific options.

```
type Direction = "north" | "south" | "east" | "west";
type DiceRoll = 1 | 2 | 3 | 4 | 5 | 6;

let dir: Direction = "north"; // OK
let roll: DiceRoll = 7; // Error!
```

VI: Literal types là các giá trị chính xác dùng làm kiểu. Hữu ích để giới hạn giá trị vào các tùy chọn cụ thể.

10. What are generics and why use them? / Generics là gì và tại sao nên dùng?

EN: Generics create reusable components that work with multiple types while maintaining type safety.

```
function identity<T>(value: T): T {
  return value;
}

identity<string>("hello"); // returns string
identity<number>(42);      // returns number
identity(true);            // inferred as boolean
```

VI: Generics tạo các thành phần tái sử dụng hoạt động với nhiều kiểu mà vẫn đảm bảo an toàn kiểu.

11. How to use generics with classes? / Cách dùng generics với classes?

EN: Generic classes can work with any type specified at instantiation.

```

class Box<T> {
    private content: T;

    constructor(value: T) { this.content = value; }
    getValue(): T { return this.content; }
}

const stringBox = new Box<string>("hello");
const numberBox = new Box(42); // inferred as Box<number>

```

VI: Generic classes có thể làm việc với bất kỳ kiểu nào được chỉ định khi khởi tạo.

12. What are generic constraints? / Generic constraints là gì?

EN: Constraints limit generic types to those having certain properties using `extends`.

```

interface HasLength { length: number; }

function logLength<T extends HasLength>(item: T): void {
    console.log(item.length); // safe - T must have length
}

logLength("hello"); // OK - string has length
logLength([1, 2, 3]); // OK - array has length
logLength(123); // Error - number has no length

```

VI: Constraints giới hạn generic types chỉ chấp nhận kiểu có các thuộc tính nhất định bằng `extends`.

13. What is Partial utility type? / Partial utility type là gì?

EN: `Partial<T>` makes all properties of `T` optional. Useful for update operations.

```
interface User { name: string; age: number; email: string; }

function updateUser(id: number, updates: Partial<User>) {
  // updates can have any subset of User properties
}

updateUser(1, { name: "John" }); // OK - only updating name
```

VI: `Partial<T>` biến tất cả properties của T thành optional. Hữu ích cho các thao tác cập nhật.

14. What are Required, Pick, and Omit utility types? / Required, Pick, và Omit là gì?

EN:

- `Required<T>` : Makes all properties required
- `Pick<T, K>` : Select specific properties
- `Omit<T, K>` : Exclude specific properties

```
interface User { name: string; age?: number; email?: string; }

type RequiredUser = Required<User>; // all required
type UserName = Pick<User, "name">; // { name: string }
type UserWithoutEmail = Omit<User, "email">; // { name, age }
```

VI: `Required` biến tất cả thành bắt buộc, `Pick` chọn properties cụ thể, `Omit` loại bỏ properties cụ thể.

15. What are Record and Readonly utility types? / Record và Readonly là gì?

EN:

- `Record<K, V>` : Creates object type with keys K and values V
- `Readonly<T>` : Makes all properties readonly

```

type Roles = "admin" | "user" | "guest";
type RolePermissions = Record<Roles, string[]>;

const permissions: RolePermissions = {
  admin: ["read", "write", "delete"],
  user: ["read", "write"],
  guest: ["read"]
};

type ReadonlyUser = Readonly<User>; // all props readonly

```

VI: Record<K, V> tạo object với keys K và values V. Readonly<T> biến tất cả properties thành chỉ đọc.

16. What is typeof type guard? / typeof type guard là gì?

EN: typeof narrows types for primitives: "string", "number", "boolean", "object", "function", "undefined", "symbol", "bigint".

```

function process(value: string | number) {
  if (typeof value === "string") {
    return value.toUpperCase(); // TS knows it's string
  }
  return value.toFixed(2); // TS knows it's number
}

```

VI: typeof thu hẹp kiểu cho primitives. TypeScript tự động hiểu kiểu trong mọi nhánh điều kiện.

17. What is instanceof type guard? / instanceof type guard là gì?

EN: instanceof checks if an object is an instance of a class. Works with class hierarchies.


```

class Dog { bark() {} }
class Cat { meow() {} }

function speak(animal: Dog | Cat) {
  if (animal instanceof Dog) {
    animal.bark(); // TS knows it's Dog
  } else {
    animal.meow(); // TS knows it's Cat
  }
}

```

VI: instanceof kiểm tra object có phải instance của class không. Hoạt động với class hierarchies.

18. What are custom type guards? / Custom type guards là gì?

EN: Custom type guards use `is` keyword to create user-defined type narrowing functions.

```

interface Fish { swim(): void; }
interface Bird { fly(): void; }

function isFish(animal: Fish | Bird): animal is Fish {
  return (animal as Fish).swim !== undefined;
}

function move(animal: Fish | Bird) {
  if (isFish(animal)) {
    animal.swim(); // TS knows it's Fish
  } else {
    animal.fly(); // TS knows it's Bird
  }
}

```

VI: Custom type guards dùng từ khóa `is` để tạo hàm thu hẹp kiểu do người dùng định nghĩa.

19. What is the difference between unknown and any? / Su khác biệt giữa unknown và any?

EN:

- any : Disables type checking, allows any operation
- unknown : Type-safe any, requires type narrowing before use

```
let a: any = "hello";  
a.foo(); // No error (but runtime error!)
```

```
let u: unknown = "hello";  
u.foo(); // Error! Must narrow first  
if (typeof u === "string") {  
  u.toUpperCase(); // OK after narrowing  
}
```

VI: any vô hiệu hóa kiểm tra kiểu. unknown an toàn hơn, bắt buộc phải thu hẹp kiểu trước khi dùng.

20. What is the never type? / never type là gì?

EN: never represents values that never occur. Used for functions that never return or impossible cases.

```

function throwError(msg: string): never {
    throw new Error(msg); // never returns
}

function infiniteLoop(): never {
    while (true) {} // never returns
}

type Shape = "circle" | "square";
function getArea(shape: Shape) {
    switch (shape) {
        case "circle": return Math.PI;
        case "square": return 1;
        default:
            const _exhaustive: never = shape; // ensures all cases handled
    }
}

```

VI: never biểu diễn giá trị không bao giờ xảy ra. Dùng cho hàm không bao giờ return hoặc trường hợp không thể xảy ra.

21. What are type assertions? / Type assertions là gì?

EN: Type assertions tell TypeScript to treat a value as a specific type. Use `as` syntax or angle brackets.

```

const input = document.getElementById("input") as HTMLInputElement;
input.value = "hello"; // OK, TS treats as HTMLInputElement

// Alternative syntax (not in JSX)
const input2 = <HTMLInputElement>document.getElementById("input");

// Double assertion for incompatible types (avoid if possible)
const x = "hello" as unknown as number;

```

VI: Type assertions bảo TypeScript xử lý giá trị như kiểu cụ thể. Dùng cú pháp `as` hoặc dấu ngoặc nhọn.

22. What is function overloading in TypeScript? / Function overloading trong TypeScript là gì?

EN: Function overloading provides multiple function signatures for different parameter types.

```
function greet(person: string): string;
function greet(persons: string[]): string[];
function greet(input: string | string[]): string | string[] {
    if (Array.isArray(input)) {
        return input.map(p => `Hello, ${p}`);
    }
    return `Hello, ${input}`;
}

greet("John");           // returns string
greet(["John", "Jane"]); // returns string[]
```

VI: Function overloading cung cấp nhiều signatures cho các kiểu tham số khác nhau.

23. How do optional and default parameters work? / Optional và default parameters hoạt động như thế nào?

EN: Optional parameters use `?`, default parameters use `=`. Optional must come after required.

```
function greet(name: string, greeting?: string): string {
    return `${greeting || "Hello"}, ${name}`;
}

function greetWithDefault(name: string, greeting = "Hello"): string {
    return `${greeting}, ${name}`;
}

greet("John");           // "Hello, John"
greet("John", "Hi");     // "Hi, John"
greetWithDefault("John"); // "Hello, John"
```

VI: Optional dung ? , default dung = . Optional phai dat sau required parameters.

24. What is the rest parameter type? / Rest parameter type la gi?

EN: Rest parameters collect multiple arguments into an array. Use ... syntax with array type.

```
function sum(...numbers: number[]): number {  
    return numbers.reduce((a, b) => a + b, 0);  
}
```

```
sum(1, 2, 3, 4); // 10
```

```
// With tuple for mixed types
```

```
function log(message: string, ...values: (string | number)[]): void {  
    console.log(message, ...values);  
}
```

VI: Rest parameters gom nhieu arguments thanh mang. Dung cu phap ... voi kieu array.

25. What are access modifiers in TypeScript classes? / Access modifiers trong TypeScript classes la gi?

EN:

- public : Accessible everywhere (default)
- private : Only within the class
- protected : Within class and subclasses

```

class Person {
    public name: string;           // accessible everywhere
    private ssn: string;           // only in Person
    protected birthDate: Date;    // in Person and subclasses

    constructor(name: string, ssn: string) {
        this.name = name;
        this.ssn = ssn;
    }
}

```

VI: public truy cap moi noi (mac dinh), private chi trong class, protected trong class va subclasses.

26. What are abstract classes? / Abstract classes là gì?

EN: Abstract classes cannot be instantiated directly. They define a blueprint with abstract methods that subclasses must implement.

```

abstract class Shape {
    abstract getArea(): number; // must be implemented

    describe(): string { // can have concrete methods
        return `Area: ${this.getArea()}`;
    }
}

class Circle extends Shape {
    constructor(private radius: number) { super(); }

    getArea(): number {
        return Math.PI * this.radius ** 2;
    }
}

```

VI: Abstract classes không thể khởi tạo trực tiếp. Chúng định nghĩa blueprint với abstract methods mà subclasses phải implement.

27. What is the difference between implements and extends? / Su khac biet giua implements va extends?

EN:

- extends : Inherits from a class (single inheritance)
- implements : Promises to fulfill an interface contract (multiple interfaces)

```
interface Flyable { fly(): void; }
interface Swimmable { swim(): void; }

class Animal { breathe() {} }

class Duck extends Animal implements Flyable, Swimmable {
  fly() { console.log("Flying"); }
  swim() { console.log("Swimming"); }
}
```

VI: extends ke thua tu class (don ke thua), implements cam ket thuc hien interface contract (co the nhieu interfaces).

28. What are mapped types? / Mapped types la gi?

EN: Mapped types create new types by transforming properties of existing types.

```
type Readonly<T> = { readonly [K in keyof T]: T[K] };
type Optional<T> = { [K in keyof T]?: T[K] };
type Nullable<T> = { [K in keyof T]: T[K] | null };

interface User { name: string; age: number; }
type ReadonlyUser = Readonly<User>;
// { readonly name: string; readonly age: number; }
```

VI: Mapped types tao kieu moi bang cach bien doi properties cua kieu hien co.

29. What are conditional types? / Conditional types là gì?

EN: Conditional types select types based on conditions using `extends` and ternary syntax.

```
type IsString<T> = T extends string ? true : false;
type A = IsString<string>; // true
type B = IsString<number>; // false

type Flatten<T> = T extends Array<infer U> ? U : T;
type Str = Flatten<string[]>; // string
type Num = Flatten<number>; // number
```

VI: Conditional types chọn kiểu dựa trên điều kiện sử dụng `extends` và cú pháp ternary.

30. What are template literal types? / Template literal types là gì?

EN: Template literal types create string types using template string syntax.

```
type EventName = "click" | "focus" | "blur";
type EventHandler = `on${Capitalize<EventName>}`;
// "onClick" | "onFocus" | "onBlur"

type HTTPMethod = "GET" | "POST" | "PUT" | "DELETE";
type Endpoint = `/${string}`;
type APIRoute = `${HTTPMethod} ${Endpoint}`;
// "GET /users", "POST /items", etc.
```

VI: Template literal types tạo kiểu string sử dụng cú pháp template string. Hữu ích để tạo các kiểu string có cấu trúc.

31. How to use infer keyword in conditional types? / Cách dùng từ khóa infer trong conditional types?

EN: `infer` extracts types from within other types in conditional type expressions.

```
// Extract return type of a function
type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;

type Fn = () => string;
type Result = ReturnType<Fn>; // string

// Extract array element type
type ElementType<T> = T extends (infer E)[] ? E : T;
type Elem = ElementType<number[]>; // number

// Extract Promise resolved type
type Awaited<T> = T extends Promise<infer U> ? Awaited<U> : T;
type Data = Awaited<Promise<Promise<string>>>; // string
```

VI: `infer` trích xuất kiểu từ bên trong các kiểu khác trong biểu thức conditional type.

32. What are distributive conditional types? / Distributive conditional types là gì?

EN: Conditional types distribute over union types when the checked type is a naked type parameter.

```

type ToArray<T> = T extends any ? T[] : never;

// Distributes over union
type Result = ToArray<string | number>;
// = ToArray<string> | ToArray<number>
// = string[] | number[]

// Prevent distribution with tuple
type ToArrayNonDist<T> = [T] extends [any] ? T[] : never;
type Result2 = ToArrayNonDist<string | number>;
// = (string | number)[]

// Practical: Exclude types from union
type Exclude<T, U> = T extends U ? never : T;
type WithoutNull = Exclude<string | null | undefined, null | undefined>;
// = string

```

VI: Conditional types phan phoi qua union types khi kieu kiem tra la type parameter don.

33. How to create recursive conditional types? / Cách tạo recursive conditional types?

EN: Recursive conditional types can process nested structures by calling themselves.

```

// Deep readonly
type DeepReadonly<T> = T extends object
  ? { readonly [K in keyof T]: DeepReadonly<T[K]> }
  : T;

type Nested = { a: { b: { c: number } } };
type ReadonlyNested = DeepReadonly<Nested>;
// { readonly a: { readonly b: { readonly c: number } } }

// Flatten deeply nested arrays
type DeepFlatten<T> = T extends (infer E)[]
  ? DeepFlatten<E>
  : T;

type Flat = DeepFlatten<number[][][]>; // number

// Get all property paths
type Paths<T> = T extends object
  ? { [K in keyof T]: K extends string
    ? T[K] extends object ? `${K}.${Paths<T[K]>}` : K
    : never }[keyof T]
  : never;

```

VI: Recursive conditional types có thể xử lý cấu trúc lồng nhau bằng cách gọi chính nó.

34. How to use template literal types for string manipulation? / Cách dùng template literal types để thao tác string?

EN: Template literal types create string patterns and can transform string literals.

```

// Event handler names
type EventName = "click" | "focus" | "blur";
type Handler = `on${Capitalize<EventName>}`;
// "onClick" | "onFocus" | "onBlur"

// CSS property to JS property
type CSSProp = "background-color" | "font-size";
type CamelCase<S extends string> = S extends `${infer A}-${infer B}`
  ? `${A}${Capitalize<CamelCase<B>>}`
  : S;
type JSProp = CamelCase<CSSProp>; // "backgroundColor" | "fontSize"

// Intrinsic string manipulation types
type Upper = Uppercase<"hello">; // "HELLO"
type Lower = Lowercase<"HELLO">; // "hello"
type Cap = Capitalize<"hello">; // "Hello"
type Uncap = Uncapitalize<"Hello">; // "hello"

```

VI: Template literal types tạo các pattern string và có thể biến đổi string literals.

35. What are mapped types with as clause? / Mapped types với mệnh đề as là gì?

EN: The `as` clause in mapped types allows key remapping, filtering, and transformation.

```

// Rename keys
type Getters<T> = {
    [K in keyof T as `get${Capitalize<K & string>}`]: () => T[K]
};
type Person = { name: string; age: number };
type PersonGetters = Getters<Person>;
// { getName: () => string; getAge: () => number }

// Filter keys
type OnlyStrings<T> = {
    [K in keyof T as T[K] extends string ? K : never]: T[K]
};
type Mixed = { name: string; age: number; email: string };
type StringProps = OnlyStrings<Mixed>;
// { name: string; email: string }

// Remove keys
type RemoveKind<T> = { [K in keyof T as Exclude<K, "kind">]: T[K] };

```

VI: Menh de as trong mapped types cho phép anh xa lai key, loc, va bien doi.

36. How to create type-safe event emitters with template literals? / Cách tạo event emitters an toàn kiểu voi template literals?

EN: Combine template literal types with mapped types for type-safe event handling.

```

type Events = {
  userCreated: { id: string; name: string };
  userDeleted: { id: string };
  orderPlaced: { orderId: string; amount: number };
};

type EventEmitter<E> = {
  on<K extends keyof E>(
    event: K,
    handler: (data: E[K]) => void
  ): void;
  emit<K extends keyof E>(event: K, data: E[K]): void;
};

const emitter: EventEmitter<Events> = createEmitter();

// Type-safe!
emitter.on("userCreated", (data) => {
  console.log(data.name); // data is { id: string; name: string }
});

emitter.emit("orderPlaced", { orderId: "123", amount: 99 }); // OK
emitter.emit("orderPlaced", { id: "123" }); // Error!

```

VI: Ket hop template literal types voi mapped types de xu ly event an toan kieu.

37. What are const assertions and when to use them? / Const assertions la gi va khi nao nen dung?

EN: as const makes values deeply readonly and infers the narrowest literal types.

```

// Without as const – types are widened
const config = { api: "/users", method: "GET" };
// { api: string; method: string }

// With as const – literal types preserved
const config2 = { api: "/users", method: "GET" } as const;
// { readonly api: "/users"; readonly method: "GET" }

// Arrays become readonly tuples
const tuple = [1, "hello", true] as const;
// readonly [1, "hello", true]

// Extract literal union from array
const STATUS = ["pending", "active", "completed"] as const;
type Status = typeof STATUS[number];
// "pending" | "active" | "completed"

// Use with function parameters
function request(url: string, method: "GET" | "POST") {}
const req = { url: "/api", method: "GET" } as const;
request(req.url, req.method); // OK – method is "GET", not string

```

VI: `as const` làm giá trị thành readonly sau và suy luận kiểu literal hẹp nhất.

38. What is the satisfies operator and how is it different from type annotation? / Toán tử satisfies là gì và khác type annotation như thế nào?

EN: `satisfies` validates a value matches a type while preserving the inferred narrower type.

```
type Colors = Record<string, [number, number, number] | string>;
```

```
// Type annotation – loses specific info
```

```
const palette1: Colors = {
```

```
  red: [255, 0, 0],
```

```
  green: "#00ff00"
```

```
};
```

```
palette1.red.map(x => x); // Error! Could be string
```

```
// satisfies – validates but keeps narrow types
```

```
const palette2 = {
```

```
  red: [255, 0, 0],
```

```
  green: "#00ff00"
```

```
} satisfies Colors;
```

```
palette2.red.map(x => x); // OK – known to be array
```

```
palette2.green.toUpperCase(); // OK – known to be string
```

```
// Combine with as const
```

```
const routes = {
```

```
  home: "/",
```

```
  about: "/about"
```

```
} as const satisfies Record<string, string>;
```

```
// Readonly + validated
```

VI: satisfies xác nhận giá trị khớp kiểu nhưng giữ kiểu suy luận hẹp hơn, khác với type annotation sẽ mở rộng kiểu.

39. How to combine as const with satisfies? / Cách kết hợp as const với satisfies?

EN: as const satisfies Type gives you both immutability and type validation.


```

// Define allowed routes
type Routes = Record<string, `/${string}`>;

const routes = {
  home: "/",
  users: "/users",
  profile: "/users/profile"
} as const satisfies Routes;

// Benefits:
// 1. Values are readonly
// 2. Validated against Routes type
// 3. Literal types preserved

type RouteKeys = keyof typeof routes; // "home" | "users" | "profile"
type HomeRoute = typeof routes.home;   // "/" (not string)

// Error if invalid
const badRoutes = {
  home: "/",
  invalid: "no-slash" // Error! Must start with /
} as const satisfies Routes;

```

VI: as const satisfies Type cho ca tinh bat bien va xac nhan kieu.

40. What is module augmentation? / Module augmentation là gì?

EN: Module augmentation extends existing modules with additional declarations without modifying original code.

```
// Extending Express Request
import { Request } from "express";

declare module "express" {
  interface Request {
    user?: { id: string; role: "admin" | "user" };
    sessionId?: string;
  }
}

// Now available in handlers
app.get("/", (req: Request, res) => {
  console.log(req.user?.role);
  console.log(req.sessionId);
});

// Extending third-party library
declare module "lodash" {
  interface LoDashStatic {
    customMethod<T>(arr: T[]): T;
  }
}
```

VI: Module augmentation mô phỏng modules hiện có với khai báo thêm mà không sửa code gốc.

41. What is declaration merging? / Declaration merging là gì?

EN: Declaration merging combines multiple declarations with the same name into a single definition.

```

// Interface merging
interface Box { width: number; }
interface Box { height: number; }
// Result: interface Box { width: number; height: number; }

// Namespace + Class merging
class Album { label: Album.AlbumLabel = { name: "" }; }
namespace Album {
  export interface AlbumLabel { name: string; }
}
const album = new Album();
album.label.name = "Epic";

// Namespace + Enum merging
enum Color { Red, Green, Blue }
namespace Color {
  export function mixColors(c1: Color, c2: Color): Color {
    return c1 + c2;
  }
}
Color.mixColors(Color.Red, Color.Blue);

```

VI: Declaration merging kết hợp nhiều khai báo cùng tên thành một định nghĩa.

42. How to augment global types? / Cách mở rộng global types?

EN: Use `declare global` to add types to global scope in a module file.

```
// Add to Window
declare global {
  interface Window {
    analytics: { track(event: string): void };
    __APP_VERSION__: string;
  }

  // Add global variable
  var DEBUG_MODE: boolean;

  // Extend built-in types
  interface Array<T> {
    customFilter(fn: (item: T) => boolean): T[];
  }
}

// Usage
window.analytics.track("pageview");
console.log(window.__APP_VERSION__);
globalThis.DEBUG_MODE = true;

// Must export to make it a module
export {};
```

VI: Dùng `declare global` để thêm kiểu vào phạm vi global trong file module.

43. What are advanced type guard patterns? / Các pattern type guard nâng cao là gì?

EN: Advanced type guards handle complex scenarios with discriminated unions and assertion functions.

```

// Discriminated union guard
type ApiResponse<T> =
  | { status: "success"; data: T }
  | { status: "error"; error: string };

function isSuccess<T>(res: ApiResponse<T>): res is { status: "success"; data: T } {
  return res.status === "success";
}

// Assertion function
function assertNonNull<T>(val: T): asserts val is NonNullable<T> {
  if (val === null || val === undefined) throw new Error("Null!");
}

// Type predicate with generics
function hasProperty<T, K extends string>(
  obj: T,
  key: K
): obj is T & Record<K, unknown> {
  return typeof obj === "object" && obj !== null && key in obj;
}

const data: unknown = { name: "John" };
if (hasProperty(data, "name")) {
  console.log(data.name); // OK
}

```

VI: Type guards nâng cao xử lý các tình huống phức tạp với discriminated unions và assertion functions.

44. How to implement exhaustive checking with discriminated unions? / Cách implement kiểm tra exhaustive với discriminated unions?

EN: Use the `never` type to ensure all union cases are handled at compile time.

```

type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; side: number }
  | { kind: "triangle"; base: number; height: number };

function assertNever(x: never): never {
  throw new Error(`Unexpected: ${JSON.stringify(x)}`);
}

function getArea(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      return Math.PI * shape.radius ** 2;
    case "square":
      return shape.side ** 2;
    case "triangle":
      return 0.5 * shape.base * shape.height;
    default:
      return assertNever(shape); // Compile error if case missing
  }
}

// If you add a new shape kind, TypeScript will error
// until you handle it in the switch

```

VI: Dung kieu `never` de dam bao tat ca truong hop union duoc xu ly luc compile.

45. What are discriminated unions best practices? / Cac best practices cho discriminated unions la gi?

EN: Best practices for effective discriminated unions:

```

// 1. Use literal type for discriminant
type Result<T, E> =
  | { ok: true; value: T }
  | { ok: false; error: E };

// 2. Make discriminant the first property
type Action =
  | { type: "INCREMENT"; amount: number }
  | { type: "DECREMENT"; amount: number }
  | { type: "RESET" };

// 3. Use const enum for discriminant values
const enum ActionType {
  Fetch = "FETCH",
  Success = "SUCCESS",
  Error = "ERROR"
}

// 4. Create type-safe action creators
function createAction<T extends Action["type"]>(
  type: T,
  payload: Extract<Action, { type: T }> extends { amount: number }
    ? number
    : never
) { /* ... */ }

// 5. Use Result type for error handling
function divide(a: number, b: number): Result<number, string> {
  if (b === 0) return { ok: false, error: "Division by zero" };
  return { ok: true, value: a / b };
}

```

VI: Best practices: dùng literal type cho discriminant, dat discriminant dau tien, dùng const enum, tao action creators an toan kieu.

46. What is covariance in TypeScript? / Covariance trong TypeScript là gì?

EN: Covariance means subtype relationship is preserved in the same direction. Applies to output/return positions.

```
class Animal { name = ''; }
class Dog extends Animal { bark() {} }

// Covariant: Dog[] assignable to Animal[] (read position)
const dogs: Dog[] = [new Dog()];
const animals: readonly Animal[] = dogs; // OK

// Function return types are covariant
type Producer<T> = () => T;
const produceDog: Producer<Dog> = () => new Dog();
const produceAnimal: Producer<Animal> = produceDog; // OK

// Generic with out modifier (TS 4.7+)
type Getter<out T> = () => T;
// Explicitly marks T as covariant (output only)
```

VI: Covariance nghĩa là quan hệ subtype được giữ nguyên cùng hướng. Áp dụng cho vị trí output/return.

47. What is contravariance in TypeScript? / Contravariance trong TypeScript là gì?

EN: Contravariance means subtype relationship is reversed. Applies to input/parameter positions.


```

class Animal { name = ''; }
class Dog extends Animal { bark() {} }

// Contravariant: Handler<Animal> assignable to Handler<Dog>
type Handler<T> = (item: T) => void;

const handleAnimal: Handler<Animal> = (a) => console.log(a.name);
const handleDog: Handler<Dog> = handleAnimal; // OK with strictFunctionTypes

// Why? A function that can handle any Animal can handle Dogs
function processAll(handler: Handler<Dog>) {
  handler(new Dog());
}
processAll(handleAnimal); // Safe – Animal handler works for Dog

// Generic with in modifier (TS 4.7+)
type Consumer<in T> = (item: T) => void;
// Explicitly marks T as contravariant (input only)

```

VI: Contravariance nghĩa là quan hệ subtype bị đảo ngược. Áp dụng cho vị trí input/parameter.

48. What are the important strict flags in tsconfig? / Cac strict flags quan trọng trong tsconfig là gì?

EN: Key strict compiler options for type safety:

```

{
  "compilerOptions": {
    "strict": true,                // Enables all strict checks
    "strictNullChecks": true,      // null/undefined are distinct types
    "strictFunctionTypes": true,   // Strict function type checking
    "strictBindCallApply": true,   // Strict bind/call/apply
    "strictPropertyInitialization": true, // Class props must be initialized
    "noImplicitAny": true,         // Error on implicit any
    "noImplicitThis": true,        // Error on implicit this
    "useUnknownInCatchVariables": true, // catch(e) is unknown, not any
    "noUncheckedIndexedAccess": true, // array[i] is T | undefined
    "exactOptionalPropertyTypes": true // Distinguish missing vs undefined
  }
}

```

```

// noUncheckedIndexedAccess example
const arr = [1, 2, 3];
const val = arr[0]; // number | undefined (must check!)

```

```

// exactOptionalPropertyTypes example
interface Config { timeout?: number; }
const c1: Config = { timeout: undefined }; // Error with exact optional!
const c2: Config = {}; // OK – property missing

```

VI: Các strict flags quan trọng: `strict` bắt tất cả, `strictNullChecks` phân biệt null, `noUncheckedIndexedAccess` kiểm tra index an toàn.

49. How to optimize TypeScript compilation performance? / Cách tối ưu hiệu suất biên dịch TypeScript?

EN: Tips for faster TypeScript compilation:

```

{
  "compilerOptions": {
    "incremental": true,          // Reuse previous compilation
    "tsBuildInfoFile": ".tsbuildinfo",
    "skipLibCheck": true,        // Skip .d.ts checking
    "isolatedModules": true      // Faster with bundlers
  },
  "include": ["src/**/*"],       // Limit scope
  "exclude": ["node_modules", "dist"]
}

```

```

// 1. Use interface over type for object shapes (faster)
interface User { name: string; } // Preferred
type User = { name: string; };    // Slower for complex types

// 2. Avoid deep type instantiation
type DeepPartial<T> = { [K in keyof T]?: DeepPartial<T[K]> };
// Limit recursion depth in complex types

// 3. Use const enum for zero runtime cost
const enum Status { Active, Inactive }
// Inlined at compile time

// 4. Split large codebases with project references
// tsconfig.json in each package with composite: true

```

VI: Tips: dùng `incremental`, `skipLibCheck`, interface thay type, const enum, project references cho codebases lớn.

50. What are project references and when to use them? / Project references là gì và khi nào nên dùng?

EN: Project references split large codebases into smaller units for faster builds and better organization.

```

// tsconfig.json (root)
{
  "references": [
    { "path": "./packages/core" },
    { "path": "./packages/api" },
    { "path": "./packages/web" }
  ]
}

// packages/core/tsconfig.json
{
  "compilerOptions": {
    "composite": true,          // Required for references
    "declaration": true,       // Generate .d.ts
    "declarationMap": true,    // Source maps for declarations
    "outDir": "./dist"
  }
}

// packages/api/tsconfig.json
{
  "compilerOptions": { "composite": true },
  "references": [
    { "path": "../core" }      // Depends on core
  ]
}

# Build all projects in dependency order
tsc --build

# Build specific project
tsc --build packages/api

# Watch mode with incremental builds
tsc --build --watch

```

VI: Project references chia codebases lớn thành đơn vị nhỏ hơn để build nhanh hơn và tốt chức năng hơn.